Task 2: Contracts, nested data and a star schema

CSEN1095 Data Engineering · W26 · Graded task · follows Labs 4, 5 and 6

Deadline: Friday 13 November 2026, 11:59 PM

The situation

The student clubs from Task 1 now sell tickets to their events through an app. The app sends every booking as one JSON document per line, in `bookings.jsonl` (283 lines). Before the clubs see a single revenue number, the data has to be checked, flattened and modelled.

Your job: keep only the bookings that meet the contract, turn the nested documents into flat tables, build a star schema, and save it as Parquet.

The data

One line looks like this (formatted here for reading):

```
{
  "booking_id": "B0002",
  "booked_at": "2026-09-27T12:29:00Z",
  "student": {"id": "61-3826", "name": "Laila Adel", "faculty": "IET"},
  "event": {"id": "EV-01", "title": "Robotics Demo Day", "club": "Robotics",
            "date": "2026-10-15", "venue": "Hall A"},
  "tickets": [{"type": "standard", "qty": 1, "price": 40.0}],
  "total": 40.0,
  "payment": "card",
  "status": "confirmed"
}
```

Field	Meaning
`booking_id`	The id the app gave the booking
`booked_at`	When the student booked, in UTC
`student`	Who booked: GUC id, name, faculty
`event`	The event: id, title, the club running it, its date and venue
`tickets`	One entry per ticket type in the booking: `type`, `qty`, `price` (EGP per ticket)
`total`	What the student paid for the whole booking
`payment`	`card`, `cash` or `wallet`

Field	Meaning
`status`	`confirmed` or `cancelled`

What to do

Work in a notebook, `task2.ipynb`. Leave `bookings.jsonl` unchanged, and do everything in code.

Your first code cell must set the input file:

```
DATA = "bookings.jsonl"
```

and every later cell must read from `DATA`. We will run your notebook a second time with `DATA` pointing to a **different file with the same shape and different problems**. Your counts must come from your code, not be typed in.

Part A: The contract (Lab 4)

1. **Write the contract as Pydantic models**: one for a ticket, one for the student, one for the event, and one for the whole booking that uses the other three. A valid booking meets **every** rule below:

Field	Rule
`student.id`	Two digits, a dash, four digits, e.g. `58-1234`
`student.faculty`	One of `MET`, `IET`, `EMS`, `Pharmacy`, `Management`
`event`	Present, with all five fields; `club` one of `Robotics`, `Debate`, `Music`, `Football`, `Chess`
`tickets`	At least one ticket
each ticket	`type` one of `standard`, `vip`, `student`; `qty` at least 1; `price` not negative
`total`	Equal to the sum of `qty × price` over the tickets
`booked_at`	Before the event's date: nobody books a ticket after the event
`payment`, `status`	One of the values in the table above

   The last two rules involve more than one field. Use a `model_validator`.

2. **Validate every line.** A booking that fails goes to a **quarantine** with its `booking_id` and the reason, and is not used again. Save the quarantine as `quarantine.jsonl`.

3. **Re-sent bookings.** The app sometimes sends the same booking twice. Keep the first copy of each `booking_id`, and quarantine the others with the reason `duplicate booking_id`.

4. **Report.** Show how many lines you read, how many passed, and how many were quarantined **for each reason**.

Part B: Flatten (Lab 6)

1. From the valid bookings, build two flat tables:

- **`bookings`**: one row per booking. The `student` and `event` objects become columns (`student_id`, `event_club`, …). No `tickets` column.
- **`booking_tickets`**: one row per ticket entry, with `booking_id`, `type`, `qty`, `price`, and `amount` = `qty × price`.

2. **Check, with assertions:** `booking_id` is unique in `bookings`, and for every booking the `amount`s of its tickets add up to its `total`.
3. **Schema drift.** Some documents have a field that the example above doesn't. Find it, and find from when it appears.

Part C: Model and store (Lab 5)

1. In **DuckDB**, build a star schema with **one row per ticket entry** as the fact:
   - `dim_event`: one row per event, with a surrogate key `event_key`;
   - `dim_student`: one row per student, with a surrogate key `student_key`;
   - `fact_ticket_sales`: `booking_id`, `event_key`, `student_key`, `ticket_type`, `qty`, `price`, `amount`, `status`, and a date key for `booked_at` in the form `YYYYMMDD`.
2. **Check, with assertions:** the fact has as many rows as `booking_tickets`, and the sum of `amount` in the fact equals the sum of `total` in `bookings`.
3. **Answer from the star:** for **confirmed** bookings only, the number of tickets sold and the revenue **per club and ticket type**.
4. **Save as Parquet**: each dimension as one file, and the fact **partitioned by the month the booking was made** (`booked_month=…` folders), all in a folder named `star/`.

Part D: Explain what you did

In your repository's `README.md`, write **one paragraph (about 150–250 words)**. It must cover:

- how many lines you read, how many were quarantined **for each reason**, and how many bookings are left;
- why the duplicate check comes **after** validation, and not before. Would the counts change if you swapped them?
- which field drifted, from when, and what your code does with documents that don't have it;
- why the booking's `total` is **not** a column of `fact_ticket_sales`. What would `SUM(total)` over the fact give you, and why?

What to submit

A GitHub repository named:

```
{first-second}_{id}_{major}-{lab#}_task2
```

For example: **`mohamed-ahmed_58-1234_MET-01_task2`**

It should contain:

```
mohamed-ahmed_58-1234_MET-01_task2/
├── task2.ipynb          runs top to bottom without errors, outputs visible
```

```
├── bookings.jsonl       the file you were given, unchanged
├── quarantine.jsonl     your quarantined bookings, with reasons
├── star/                your Parquet files: two dimensions and the partitioned fact
├── requirements.txt     the packages your notebook needs, with versions
└── README.md            your name, ID, and the Part D paragraph
```

Add `de-guc` as a collaborator: on GitHub, open the repository → **Settings** → **Collaborators** → **Add people** → `de-guc`. The repository may be private, but without this invitation we can't see it, and it can't be graded.

Submit the repository's URL through the submission form: **https://forms.gle/V9BmtJW1K7qK6RKL8**

A repository that isn't submitted through the form isn't graded, even if `de-guc` was added.

The deadline is **Friday 13 November 2026, 11:59 PM**, for both the form and the repository. The last commit before the deadline is the one graded.

How it's graded

	Points
Part A: The contract	
Pydantic models cover every rule, including the two cross-field rules	20
Quarantine with reasons, duplicates handled, counts per reason reported	10
Part B: Flatten	
`bookings` and `booking_tickets` at the right grains, with both assertions	15
The drifting field found, with the date it appears	5
Part C: Model and store	
Star schema with surrogate keys, and both assertions	15
The answer per club and ticket type, from the star	5
Parquet: two dimension files and the fact partitioned by booking month	5
Hidden file	
The notebook runs on our second file and its counts are right	10
Part D: The paragraph	
Covers every point above, with correct numbers	10
Submission	
Repository named correctly, `de-guc` added, URL submitted through the form, notebook runs top to bottom	5
Total	**100**

Rules

- **Every fix is code.** Editing `bookings.jsonl` by hand gets zero for that part.
- **No hard-coded counts.** A number typed into the notebook instead of computed fails on the hidden file.
- **Your own work.** You must be able to explain every line of your notebook if asked.